CLAUDE.md — Asset Allocation Recommender (v4)

Phase 1: Recommendation Engine | Phase 2: Chatbot-Based Rebalancing

> This file supersedes the v1 Streamlit-era CLAUDE.md and the v2/v3 updates for architecture decisions. It is the single source of truth for all five module owners. Any change to a schema, endpoint, or config value listed here must be agreed by all five owners before merging, because every other module is coded against these exact contracts in isolation.

---

0. Change Log — this revision

#	Change	Why
1	Stock breakdown gets a **pie chart** (weight per holding) + a **company-health KPI panel** (26 KPIs across Income Statement / Balance Sheet / Cash Flow / Hybrid-Valuation) per stock	Investors currently see only a % number with no fundamentals context
2	**Caching layer**: bulk data downloaded once and reused; universe capped at **10–15 instruments** (stocks + MFs combined) for the prototype, architected to scale to 100+ later without code changes	Avoid re-hitting yfinance/AMFI/MoneyControl on every request; keep prototype fast to build/demo
3	**Correlation-matrix-based universe selection**: within a user's chosen category (sector / MF category), drop near-duplicate highly-correlated names and keep the most diversified, best-fundamentals representative	Feeds the optimizer a genuinely diversified candidate set instead of 5 clones of the same bet
4	**Chatbot** rewritten around strict NL → structured-intent extraction, sent to backend only as validated parameters (never free text)	"High precision" requirement — no hallucinated numbers reaching the optimizer
5	**Transformer forecaster locked to** `forecaster.py` (the attached, tested reference implementation) — this **replaces** the d_model=64 / 2–3 layer / 252-trading-day spec in the original synopsis	The attached config is empirically tuned and already guards against the blow-up failure mode described in its own header comment
6	**MoneyControl scraper** for per-MF portfolio holdings %, rendered as a pie chart on the frontend	Original spec only had FD/MF/Stock split; this adds "what's actually inside the mutual fund"
7	**Frontend rebuilt in React** (Vite + Tailwind + Recharts), Streamlit MVP kept only as an internal reference, not shipped	Team needs a component-based frontend that 1 person can own independently and that 4 other people can build against via a fixed API contract

Everything **not** listed above (Black-Litterman, tax engine, four-agent design, Pydantic/Jinja2 explainability guardrail, scope boundaries) is **unchanged** from the original synopsis and still applies.

0.1 Workload Rebalancing (this revision, v3)

The v2 module map bundled the tax engine and the correlation-based universe selector into the same person as the Transformer forecaster and Black-Litterman optimizer — that person's scope was disproportionately large next to, say, the chatbot owner. This revision **narrows M2 to forecasting and optimization only** and redistributes everything else evenly:

Role	v2 owner	v3 owner	Why moved
`forecaster.py` (locked config)	M2	**M2**	stays — this is the core of the role
`optimizer.py` (Black-Litterman + QP)	M2	**M2**	stays — inseparable from the forecaster's output
`tax_engine.py` (post-tax real return)	M2	**M3**	pure deterministic logic; sits naturally next to the orchestration layer that calls it between forecasting and optimizing
`universe_selector.py` (correlation dedup)	M2	**M4**	pairs naturally with the chatbot owner's existing job of turning fuzzy/ambiguous input into a clean, validated set of parameters — both modules are "filter noisy input into a precise structured output"
Fundamentals → `KpiCard[]` badge formatting	unassigned	**M3**	same owner as `/fundamentals/{ticker}`'s endpoint wiring and schema production

Net effect: **M1 = Frontend only. M2 = Forecaster + Optimizer only.** The remaining work — backend/schemas/orchestration, tax engine, fundamentals formatting, chatbot, and correlation-based universe selection — is split evenly across **M3, M4, M5**, each ending up with one clearly-scoped "intelligence/data" concern plus its original core job. See the revised ownership map in §11 and the exact cross-module call order in §7.1.

0.2 Phase Plan (this revision, v4)

The project ships in **two phases**. Phase 1 is a complete, demoable recommendation engine — capital in, allocation + pie charts + fundamentals out. Phase 2 adds the ability to talk to it and have it rebalance. **Nobody starts Phase 2 work until Phase 1 is merged, integrated, and demoable end-to-end.**

Phase 1 — Recommendation Engine (build this first)

Everything covered earlier in this file **except** the chatbot itself. Concretely, in scope:

- Intake form → `UserProfile` (§4.1)
- Data caching, bulk-once pull, T-1 snapshot refresh, 10–15 instrument prototype universe (§6.1)
- Correlation-based universe selection (§6.2) — **note:** this stays in Phase 1 even though its owner (M4) is otherwise a Phase 2 person; it's needed to build the very first recommendation, not just to rebalance one later
- MoneyControl MF holdings scraper + pie chart (§8)

- Locked Transformer forecaster (§7) + Black-Litterman optimizer (§7.1)
- Post-tax real-return tax engine (§3.1.1 of the original synopsis, owned by M3)
- Stock fundamentals → 26-KPI badge panel + pie chart (§5)
- Pydantic + Jinja2 explainability guardrail (original synopsis §3.4.1, unchanged)
- Endpoints: `POST /profile`, `POST /allocate`, `GET /fundamentals/{ticker}`, `GET /mf-holdings/{scheme}`
- Frontend screens: intake form → allocation dashboard → stock detail → MF detail. **No chat panel, no rebalance banner.**

Explicitly excluded from Phase 1: the chatbot (Ollama, intent extraction, `ChatIntent` schema), `POST /chat`, `GET /rebalance-check`, and any UI for either.

Phase 2 — Chatbot-Based Rebalancing (build after Phase 1 ships)

- Chatbot NL → structured intent extraction (§10), `ChatIntent` schema added to `/shared/schemas` at this point (not before — no need to carry an unused schema through Phase 1)
- `POST /chat` — executes one of the six functions server-side, re-runs the Phase 1 `/allocate` chain with modified constraints, returns a before/after diff
- `GET /rebalance-check/{profile_id}` — autonomous drift check against the last-known allocation. This needs a small persistence addition not covered anywhere in Phase 1: a `profile_allocations` table/store (owned by M3) recording each profile's most recent computed `AllocationExplanation`
  - timestamp, so a later check has something to diff against. Phase 1 doesn't need this because it only ever computes one allocation and shows it — nothing to compare it to yet.
- Frontend: `ChatWindow` component, diff view, rebalance-notification banner

Why the split lands this way per module

Module	Phase 1 workload	Phase 2 workload
M1 Frontend	Intake form, dashboard, stock/MF pie + KPI panels	`ChatWindow`, diff view, rebalance banner
M2 Forecaster & Optimizer	Full scope (`forecaster.py`, `optimizer.py`)	**None** — Phase 2 just calls `optimize()` again with modified constraints, no new code
M3 Backend/Schemas/Tax/Fundamentals	Core schemas, `/profile`, `/allocate`, `/fundamentals`, `/mf-holdings`, tax engine, fundamentals formatter	`ChatIntent` schema, `/chat`, `/rebalance-check`, `profile_allocations` persistence
M4 Chatbot + Universe Selection	**Only** `universe_selector.py` (needed for Phase 1's `/allocate`)	Everything chatbot: `ollama_client.py`, `intent_extractor.py`
M5 Data Caching & Scraping	Full scope, unchanged	**None** — same `cache_manager` interface serves both phases

M2 and M5 are effectively "done" once Phase 1 ships — their Phase 2 involvement is zero new code. M4 is the mirror image: light in Phase 1 (just the correlation dedup), then does the entire chatbot in Phase 2. This is intentional and fine — a phase gate, not a workload gate; M4 can help review/test other modules during the Phase 1 window if they finish early.

---

1. Project Overview (unchanged)

A data-driven asset allocation recommender for Indian retail investors, splitting capital across **FD** / **Mutual Funds** / **Stocks** using a mini Transformer forecaster, a post-tax real-return objective, and Black-Litterman portfolio construction — with an explainable, schema-verified output and a natural-language chat interface for adjustments.

Data-access constraint (unchanged): bulk historical data is pulled once for training; going forward the system only ever has **T-1 (previous trading day close)** snapshots, fetched on-demand, never live/intraday, never on a background scheduler.

---

2. Updated High-Level Architecture



Four-agent design (ProfilerAgent, ForecastAgent, OptimizerAgent, Monitor/Chatbot Agent) from the original synopsis maps onto M2 (Forecast + Optimizer agents, Phase 1) and M4 (Monitor/Chatbot agent — its "Monitor" drift-check half and "Chatbot" NL half are now both Phase 2, while M4's *other* job this revision, universe selection, is Phase 1) — ProfilerAgent is a thin function inside M3.

---

3. Repository Layout

```
/frontend                    # M1 — owns this directory exclusively
  src/components/            # AllocationPieChart, StockKpiPanel,
MfHoldingsPie, ChatWindow...
  src/api/client.ts          # typed fetch wrapper, matches §4 exactly
  src/mocks/                 # fixture JSON matching §4 response shapes, for
standalone dev

/backend
  /forecasting               # M2 — owns this directory exclusively
                              #       forecaster.py (verbatim, §7),
optimizer.py — NOTHING else
  /api                       # M3 — owns this directory exclusively
                              #       FastAPI routers, tax_engine.py,
fundamentals_formatter.py
  /chatbot                   # M4 — owns this directory
                              #       intent_extractor.py, functions.py,
ollama_client.py
  /selection                 # M4 — owns this directory too (correlation
logic, §6.2)
                              #       universe_selector.py
  /data                      # M5 — owns this directory exclusively
                              #       cache_manager.py, scrapers/ (yfinance,
mfapi, moneycontrol),
                              #       fd_rates.json, cache/ (sqlite or
parquet, gitignored)

/shared                      # jointly owned — see the freeze rule below
  /schemas                   # Pydantic models AND generated TS types — the
actual contract.
                              # Nobody hand-writes a duplicate shape anywhere
else.
  kpi_thresholds.json        # badge color/label thresholds for the 26
fundamentals KPIs (§5)

CLAUDE.md                    # this file
```

Each of the five members has their own top-level directory (or pair of directories, for M4) that nobody else writes to — this is what keeps a five-way `git merge` low-conflict. The only jointly touched path is `/shared`, which is frozen early (§12).

Rule: the `/shared/schemas` directory is the only place a response/request shape is defined. Backend imports the Pydantic models directly. Frontend imports TS types generated from them (`datamodel-code-generator` → OpenAPI → `openapi-typescript`, wired by M3, run as a `npm run gen:types` script M1 calls). This is what lets five people build in isolation and have it click together on merge day.

---

4. Shared API Contract

4.1 Core schemas (`/shared/schemas`)

python

```python
# ── Phase 1 schemas — freeze these first ──────────────────────────

class UserProfile(BaseModel):
    capital: float
    instruments: list[Literal["FD", "MF", "STOCKS"]]
    bank: str | None
    sector_pref: str | None          # stock sector filter
    mf_category_pref: str | None
    risk_thermometer: confloat(ge=0, le=100)
    tax_slab: Literal[10, 20, 30]
    horizon_years: conint(ge=1, le=40)

class AllocationExplanation(BaseModel):   # unchanged from original synopsis, §
    fd_pct: confloat(ge=0, le=100)
    mf_pct: confloat(ge=0, le=100)
    stocks_pct: confloat(ge=0, le=100)
    expected_return_post_tax: float
    return_ci_low: float
    return_ci_high: float
    risk_score: confloat(ge=0, le=100)
    sector_prefs: list[str]
    horizon_years: conint(ge=1, le=40)
    key_drivers: list[str]

class InstrumentWeight(BaseModel):        # NEW — feeds the stock/MF pie charts
    ticker_or_scheme: str
    display_name: str
    weight_pct: confloat(ge=0, le=100)

class KpiCard(BaseModel):                 # NEW — one pre-formatted KPI, badge
    label: str
    value_str: str          # e.g. "12.4%", "1.8x", "₹420 Cr"
    badge_label: str        # e.g. "Healthy", "High Leverage"
    badge_color: str        # hex, from /shared/kpi_thresholds.json
    tooltip: str

class StockFundamentals(BaseModel):       # NEW
    ticker: str
    as_of: date
    income_statement: list[KpiCard]       # 8 cards
    balance_sheet: list[KpiCard]          # 8 cards
    cash_flow: list[KpiCard]              # 2 cards
    valuation: list[KpiCard]              # 8 cards
    source: str                           # "yfinance", per §9.3 dated-sourcing

class MfHoldingsBreakdown(BaseModel):     # NEW
    scheme_name: str
    as_of_date: date
    holdings: list[InstrumentWeight]      # top holdings by % of AUM (sector or
    source_url: str
    fetched_at: datetime

# ── Phase 2 schema — add only when Phase 2 starts ─────────────────
```

```python
class ChatIntent(BaseModel):                # NEW — the ONLY thing the chatbot ma
    function: Literal[
        "adjust_allocation", "exclude_sector", "set_risk_level",
        "set_horizon", "set_tax_slab", "recompute_allocation", "clarify"
    ]
    params: dict[str, str | float | int]
    confidence: confloat(ge=0, le=1)
```

4.2 Endpoint table

Phase	Method	Path	Request	Response	Owner (orchestration cha
1	POST	`/profile`	`UserProfile`	`{profile_id}`	M3
1	POST	`/allocate`	`{profile_id}`	`AllocationExplanation` + `list[InstrumentWeight]` (per bucket)	**M3** orchestrates M4 (univ M5 (prices) → M2 (forecas (tax) → M2 (optimize) — f sequence in §7.1
1	GET	`/fundamentals/{ticker}`	—	`StockFundamentals`	**M3** builds the schema fror raw cached fundamentals, applying `/shared/kpi_threshold`
1	GET	`/mf-holdings/{scheme}`	—	`MfHoldingsBreakdown`	M3 → M5 (scraper)
2	POST	`/chat`	`{profile_id, message}`	`ChatIntent` + updated `AllocationExplanation` (diff view)	M3 → M4 (intent extractio (executes the named func runs the `/allocate` chai
2	GET	`/rebalance-check/{profile_id}`	—	`{drifted: bool, delta_pct: float, message: str}`	M3 → M2, reads/writes the `profile_allocations` s (§0.2)

All endpoints return **only** the schemas in §4.1 — no ad-hoc dict shapes. Any owner needing a new field adds it to `/shared/schemas` in its own PR and pings the other four. Phase 1 is demoable and complete with only the four Phase-1-tagged endpoints — `/chat` and `/rebalance-check` do not exist yet at that point, not even as stubs.

---

5. Update 1 — Stock Pie Chart + Company Health KPIs

- Frontend (M1): one `StockKpiPanel` component per held ticker, rendered as an accordion (first one expanded by default) — this directly ports the four KPI groups already prototyped in Streamlit (Income Statement, Balance Sheet, Cash Flow, Hybrid & Valuation), now as React cards driven purely by `StockFundamentals.income_statement / balance_sheet / cash_flow / valuation` (already-formatted `KpiCard[]`, no client-side threshold math).
- A `Recharts` `<PieChart>` above the accordion shows `InstrumentWeight[]` for the STOCKS bucket — one slice per held ticker, sized by `weight_pct`.
- **Design decision:** badge color/label thresholds (>10% net margin = "Healthy", D/E <50% = "Conservative", etc. — same cutoffs as the original Streamlit snippet) live in `/shared/kpi_thresholds.json` and are applied **once, server-side, by M3** (in `fundamentals_formatter.py`, using M5's raw cached fundamentals as input) when

building `StockFundamentals` — not duplicated in TypeScript, and not scattered across M2/M5. This avoids the two-language, two-owner business-logic drift that would otherwise happen with a five-person team.

- 26 KPIs total per stock: 8 income-statement, 8 balance-sheet, 2 cash-flow, 8 valuation — exact field list unchanged from the reference Streamlit code (Net Profit Margin, Gross Margin, EBITDA, Revenue Growth, Total Revenue, Net Income, Operating Margin, Earnings Growth / Current Ratio, Quick Ratio, D/E, Working Capital, Total Cash, Total Debt, ROE, ROA / Operating CF, Free CF / P/E, P/B, EV-EBITDA, Dividend Yield, Beta, PEG, Revenue per Employee, Payout Ratio).

---

6. Update 2 & 3 — Caching + Correlation-Based Universe Selection

6.1 Caching (M5)

- **Bulk historical pull happens once per instrument**, on first request, and is persisted locally (SQLite or Parquet under `/backend/data/cache/`, gitignored). Every later request reads from cache first; only the T-1 latest-day snapshot is re-fetched, and even that is cached with a short TTL (same trading day = cache hit).
- `cache_manager.get_prices(ticker, force_refresh=False)` is the **only** function anything else in the codebase calls to get price/NAV history — no module is allowed to call yfinance/mfapi/MoneyControl directly. This is what lets the universe scale from 15 → 100+ later by just changing a config list, with zero code changes elsewhere.
- **Prototype universe size is hard-capped at 10–15 total instruments** (mix of stocks + MF schemes) via `config/universe.yaml`. This is a config value, not a hardcoded constant, so scaling later is a one-line change plus letting the cache warm up.

6.2 Correlation-based selection (M4 — `backend/selection/universe_selector.py`) — Phase 1

Build this in Phase 1, even though M4's other file (the chatbot) is entirely Phase 2 — the recommendation engine cannot pick a universe without it, so it ships with everything else in §0.2's Phase 1 list.

1. Filter the cached universe down to the user's chosen sector / MF category.
2. Compute the pairwise return-correlation matrix (from cached daily returns, via `cache_manager.get_prices()` — M4 calls M5's interface exactly like everyone else) across the filtered candidates.
3. Where two instruments are highly correlated ($\rho > 0.85$, configurable), treat them as redundant and keep only the one with the better fundamentals/Sharpe (using `StockFundamentals` / historical Sharpe already in cache) — this stops the optimizer from being handed several near-duplicate bets dressed up as diversification.
4. The surviving, de-duplicated subset is what actually goes into the Transformer forecaster and the Black-Litterman optimizer.

- This is a lightweight, defensible stand-in for the full cross-asset-attention covariance blending described as a stretch goal in the original synopsis (§5.3) — same intent (data-driven diversification), cheaper to build for a 10–15-instrument prototype.

---

7. Update 4 — Transformer Forecaster: Locked to `forecaster.py`

`backend/engine/forecaster.py` is used **verbatim** as attached — this is the reference implementation, empirically tuned, and it **overrides** the earlier d_model=64 / 2–3-layer / TRADING_DAYS=252 description in the original project synopsis. Flagging this explicitly so nobody on the team re-implements the old spec from memory.

Locked hyperparameters (do not change without re-running the validation that produced them):

Param	Value
`TRADING_DAYS`	63
`SEQ_LEN`	20
`EPOCHS`	200
`d_model`	32
`nhead`	4
`num_layers`	1
`MU_BLEND_ALPHA`	0.3 (transformer point-forecast weight; rest is historical mean)
`MU_CLAMP_SHARPE`	1.0 (annualized mu clamped to ±1 annualized σ around historical mean)
`SIGMA_CLAMP_BAND`	(0.5, 3.0) × historical daily std
`LOG_VAR_CLAMP`	(-4.0, 4.0)
per-instrument seed	`abs(hash(instrument_name)) % (2**32)` — **critical**: never call `forecast_instrument()` with the default `seed=0` inside a loop over multiple instruments, or every instrument's tiny 200-epoch model starts from identical weights and forecasts collapse toward the same sign/magnitude (this was a diagnosed-and-patched bug in the MVP; `forecast_universe()` already does this correctly)
Device	CUDA if available (RTX 4050 6GB), else CPU — orthogonal to the sequential per-instrument loop
Loop	Strictly sequential, one instrument at a time, no concurrency across instruments (Section 6 note in the file header)

M2 owns `optimizer.py` (Black-Litterman via PyPortfolioOpt) alongside this file — **and nothing else**. `tax_engine.py` (post-tax real-return transform, unchanged from original §3.1.1) is now owned by M3 (see §0.1 for why).

7.1 Narrowed M2 scope + exact cross-module call order

M2's job is strictly: forecast, then optimize. It does not touch tax, does not filter the universe, and does not fetch data. The full `POST /allocate` sequence, orchestrated by M3, is:

```
1. M4.universe_selector.select_universe(profile, candidate_pool)
       -> list[str]                                    # deduped
tickers/schemes, §6.2

2. M5.cache_manager.get_prices(ticker) for each in (1)
       -> dict[str, pd.Series]                         # price/NAV history,
§6.1

3. M2.forecaster.forecast_universe(price_histories)
       -> dict[str, tuple[mu_nominal_annual, sigma_annual]]   # locked
config, §7

4. M3.tax_engine.apply_post_tax(mu_nominal, instrument_class,
       holding_period, tax_slab, cpi)  for each instrument
       -> dict[str, mu_post_tax]                       # original synopsis
§3.1.1

5. M2.optimizer.optimize(mu_post_tax, price_histories,
       risk_thermometer, constraints)
       -> AllocationExplanation, list[InstrumentWeight]  # Black-Litterman +
QP, §5.3
```

`optimizer.optimize()` computes the covariance matrix Σ **internally** from `price_histories` (via PyPortfolioOpt's `risk_models`) — it does not need M4's correlation matrix as an input. These are two independent, non-conflicting uses of the same underlying return data: M4's correlation matrix in step 1 decides *which instruments survive*; M2's covariance matrix in step 5 decides *how much weight the survivors get*.

M2's public contract — the only thing any other module calls:

python

```
def forecast_universe(price_histories: dict[str, pd.Series]) -> dict[str, tuple

def optimize(mu_post_tax: dict[str, float], price_histories: dict[str, pd.Serie
             risk_thermometer: float, constraints: dict) -> tuple[AllocationExp
```

Both functions are fully testable standalone with synthetic hash-seeded price series and a synthetic `mu_post_tax` dict — M2 never needs a real M3, M4, or M5 running to develop or demo this module, which is what makes it safe to build in isolation.

---

8. Update 5 — MoneyControl MF Holdings Scraper

- New scraper module: `backend/data/scrapers/moneycontrol.py` (owned by M5).
- Pulls each selected MF scheme's **portfolio holdings** % (top sector or stock-level weights, whichever MoneyControl's scheme page exposes cleanly) and returns `MfHoldingsBreakdown`.
- **Freshness cadence:** fund holdings are disclosed monthly, not daily — cache TTL is set to ~30 days, consistent with the FD-rate lookup's own philosophy (§3.5 of the original synopsis: freshness automation traded for build simplicity where the underlying data itself doesn't change fast).
- Every scrape is stamped with `source_url` and `fetched_at` and stored alongside the parsed data — this directly satisfies the "dated, source-verifiable" requirement from

the original synopsis's §9.3 anti-"just ask ChatGPT" argument.

- Respect MoneyControl's `robots.txt` and rate-limit requests (small fixed delay between calls); this is an academic prototype scraping a small, capped (10–15 instrument) universe on an infrequent cadence, not a high-volume crawler.
- Frontend (M1): `MfHoldingsPie` component renders `MfHoldingsBreakdown.holdings` as a Recharts pie, shown alongside each selected MF in the dashboard.

---

9. Update 6 — React Frontend

- Stack: **React 18 + Vite + TypeScript + Tailwind + Recharts.** The Streamlit app remains in the repo under `/reference/streamlit_mvp/` purely as a working reference for KPI-card formatting logic — it is not the shipped product.
- **Phase 1 screens:** intake form → allocation dashboard (FD/MF/Stocks pie + risk meter + post-tax return range) → stock detail (§5) → MF detail (§8). No chat panel yet.
- **Phase 2 screens:** chat panel (§10) added to the dashboard, plus a rebalance-notification banner driven by `GET /rebalance-check`.
- All data access goes through `src/api/client.ts`, typed against `/shared/schemas` (auto-generated, never hand-typed) — see §3 rule.
- M1 develops against `src/mocks/` fixtures until M3's real endpoints are live, so the frontend is demoable standalone at any point.

---

10. Chatbot — NL → Structured Intent (High Precision) — PHASE 2 ONLY

Nothing in this section is built during Phase 1. It starts only once Phase 1 is merged, integrated, and demoable (§0.2, §12).

- LLM: Ollama, local, `qwen2.5:1.5b-instruct` primary / `llama3.2:1b` latency fallback (unchanged from original §3.4) — function-calling / structured-output mode only, never free-form generation reaching the backend.
- Output is validated straight into `ChatIntent` (§4.1, added to `/shared/schemas` at the start of Phase 2). If validation fails **twice** (malformed JSON, out-of-range param, unknown function), the chatbot returns the `clarify` intent and asks the user a follow-up question rather than guessing — this is the "high precision" requirement: **wrong-but-confident is worse than asking again.**
- `ChatIntent.confidence` below a configurable threshold (default 0.7) also routes to `clarify`, even if the schema technically validated.
- The backend (M3) executes exactly one of the six functions server-side (`adjust_allocation`, `exclude_sector`, `set_risk_level`, `set_horizon`, `set_tax_slab`, `recompute_allocation`) — the LLM never touches numbers directly, it only names a function and supplies parameters, matching the original synopsis's separation of "understanding the user" (LLM) from "the financial math" (deterministic backend).
- The same `recompute_allocation()` path is called by both a user chat message and the autonomous drift check (`/rebalance-check`) — one code path, two triggers, as specified in the original synopsis §3.3.
- **New in Phase 2, owned by M3:** a `profile_allocations` store (a simple SQLite table is enough — `profile_id, allocation_json, computed_at`) recording every allocation the system has ever produced for a profile. `/rebalance-check` diffs the current re-forecast against the most recent row for that `profile_id`. Phase 1 never writes to this table because Phase 1 has no concept of "recompute" — it only ever computes once.

---

11. Module Ownership Map (5-person team, rebalanced v3, phased v4)

#	Module	Phase 1 scope	Phase 2 scope	Directory
M1	Frontend	Intake form, allocation dashboard, stock/MF pie charts + KPI panels	`ChatWindow`, diff view, rebalance banner	`/frontend`
M2	Forecaster & Optimizer	`forecaster.py` (locked, §7) + `optimizer.py` — full scope	**None** — reused as-is	`/backend/forecasting`
M3	Backend API, Schemas, Tax Engine, Fundamentals	Core schemas, `/profile`, `/allocate`, `/fundamentals`, `/mf-holdings`, `tax_engine.py`, `fundamentals_formatter.py`	`ChatIntent` schema, `/chat`, `/rebalance-check`, `profile_allocations` store	`/backend/api`, `/shared`
M4	Chatbot + Universe Selection	**Only** `universe_selector.py` (§6.2)	Everything chatbot: `ollama_client.py`, `intent_extractor.py` (§10)	`/backend/chatbot`, `/backend/selection`
M5	Data Caching & Scraping	Full scope — `cache_manager.py`, all scrapers, FD JSON	**None** — same interface serves both phases	`/backend/data`

Why this split: M1 and M2 are single-purpose, focused roles as originally requested (frontend only; forecasting/optimization only). The remaining backend surface — orchestration/schemas, tax logic, fundamentals formatting, chatbot, and universe selection — is divided evenly across M3/M4/M5 so no one non-frontend, non-forecasting member carries a disproportionate share (each of M3/M4/M5 owns one "core" concern plus one "supporting" concern, per §0.1) — and now that work is further staged across the two phases per §0.2.

Branch convention: `module/<number>-<short-name>-p1` / `-p2` (e.g. `module/2-forecaster-p1`, `module/4-chatbot-p2`). Every module PR must pass its own unit tests **and** conform to `/shared/schemas` without modifying it unilaterally.

---

12. Integration / Merge Protocol

1. **Freeze `/shared/schemas` early** (M3 drafts it in week 1 from §4 above; all five owners sign off before anyone writes business logic against it).
2. Each module ships **runnable standalone** against mocks/fixtures — no module should require another module's real code to demo its own piece. A ready-made fixture bundle covering every schema in §4.1 (sample profiles, a full allocation per profile, 26-KPI stock fundamentals for all 8 universe stocks, MF holdings, deterministic synthetic price histories for all 13 instruments, and Phase 2's chat/rebalance fixtures) ships at `/shared/fixtures/` — see `/shared/fixtures/README_DUMMY_DATA.md` for the exact per-module usage. Nobody needs to hand-roll their own mock data from scratch.
3. `make dev` at repo root (owned by M3) spins up the full stack — frontend + backend + Ollama — for integration smoke-testing once modules land on `main`.
4. PR checklist before merge: (a) conforms to `/shared/schemas` exactly, (b) no hardcoded ports/paths/API keys, (c) `.env.example` updated if new config was added, (d) module's own tests pass in CI.

5. Integration order **within Phase 1** (lowest risk first): **M5 first (true leaf, everyone else mocks it) → M2 and M4's `universe_selector.py` in parallel (both are pure-function modules, fully testable against synthetic/mocked inputs, and don't depend on each other) → M3 (wires M2 + M4 + M5 together for real, plus its own tax engine and fundamentals formatting) → M1 last (points the real frontend at M3's now-live Phase 1 endpoints).**
6. **Phase gate: do not open any Phase 2 branch until Phase 1 has been merged to `main`, `make dev` runs the full Phase 1 stack cleanly, and someone outside the team who didn't write the code can go intake → allocation → stock detail → MF detail without hitting an error.** This is the concrete, checkable trigger for "Phase 1 is done."
7. Integration order **within Phase 2**: M4's chatbot and M3's `/chat` + `/rebalance-check` + `profile_allocations` store land together (they're tightly coupled — M3 can't wire `/chat` without M4's `extract_intent()`, and M4's function is hard to demo without M3's execution layer to call), then M1's `ChatWindow` + rebalance banner last. M2 and M5 need no new PRs in this phase.

---

13. Phase Definition of Done

Phase 1 is done when, with zero chatbot code anywhere in the repo:

- A user can fill out the intake form and get back a real (not mocked) FD/MF/Stocks allocation, computed by the real locked Transformer + Black-Litterman pipeline on the real 10–15-instrument cached universe.
- The stocks bucket shows a real pie chart and real 26-KPI fundamentals panels, badges included.
- At least one selected MF shows a real MoneyControl-scraped holdings pie chart with a visible source URL and fetch date.
- `make dev` boots the whole Phase 1 stack with one command and a fresh reviewer can complete the full flow without being told how.

Phase 2 is done when, on top of a working Phase 1:

- A chat message like "move 10% from stocks to FD" produces a validated `ChatIntent`, a re-computed allocation, and a visible before/after diff in the UI.
- An ambiguous or contradictory message ("zero risk but exclude FD") reliably produces a `clarify` response instead of a wrong confident answer.
- `GET /rebalance-check` correctly reports drift for a profile whose stored allocation has diverged from a freshly recomputed one, and correctly reports no drift when it hasn't.

---

14. Unchanged Scope Boundaries (carried over from original synopsis)

No trade execution, no SEBI-regulated advice framing, no live/intraday data, no cumulative ₹1.25L LTCG exemption tracking, no old-vs-new tax regime modeling, no explicit market-regime classifier. Pydantic + Jinja2 explainability guardrail, four-agent design, and post-tax real-return objective all remain as originally specified.